



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Movie recommendation system

CORSO DI
NUMERICAL ANALYSIS FOR MACHINE LEARNING

Author: **Mauro Simone Lorenzo Meroi**

Student ID: 10767574 10788126
Advisor: Prof. Edie Miglio
Academic Year: 2024-25

Contents

Contents	i
Introduction	1
Implementation	3
Conclusion	13

Introduction

In today's data-driven landscape, **recommendation systems** play a pivotal role in enhancing user experiences across various digital platforms. These systems are designed to predict and suggest items—such as movies, products, or music—that a user is likely to appreciate based on their preferences or behavior.

A **movie recommendation system**, in particular, aims to present users with films that match their tastes and interests. By analyzing historical data—such as user ratings, watch history, and preferences—these systems personalize content delivery, reducing the effort required by users to find something engaging to watch. This personalization not only improves user satisfaction but also increases user engagement, retention, and overall platform profitability.

This project aims to develop a robust recommendation model to provide meaningful suggestions to users, while effectively addressing the critical challenge known as the **cold-start problem**.

The Cold-Start Problem

The **cold-start problem** refers to the difficulty that recommendation systems face when there is insufficient data about a new user or a new item. Since these systems often rely on user-item interactions (e.g., ratings or views), the absence of such data makes it challenging to generate accurate recommendations.

Cold-start issues are generally categorized into three types:

- **User cold-start:** Occurs when a new user joins the platform without any previous interactions. The system has little to no information about their preferences, making it difficult to personalize recommendations.
- **Item cold-start:** Happens when a new item (e.g., a newly released movie) is added to the system. Since it has not been interacted with yet, the system struggles to determine which users might be interested in it.
- **System cold-start:** Arises when a recommendation system is newly deployed and lacks sufficient user or item data across the board.

This algorithm aims to be robust to the first two kinds of cold start adopting popular techniques like *content-based filtering* and *collaborative filtering*.

Implementation

The proposed model consists of three main steps that synergistically enhance the recommendation process:

1. **Matrix Factorization using ALS**

ALS is used to decompose the user-item rating matrix into hidden feature matrices, capturing hidden patterns and getting a first approximation of what could be the predicted ratings.

2. **User Partitioning through Demographic Similarities**

Users are grouped based on demographic data (age, sex, preferred genres) to enhance cold-start recommendations. From this step we will get to a second prediction of ratings based on giving to similar user similar rating of the films.

3. **Enhanced Movie Similarity Calculations**

Text features and Word2Vec embeddings are used to calculate contextual similarities between movies. In this way we will compute with a greater accuracy similar films. From this step we will get another prediction based on the fact that similar

Step 0 - Data Wrangling

In this preliminary step, we worked with two different datasets: one containing approximately 100,000 ratings and another with 1 million ratings. Each dataset includes three main components:

- A dataframe containing user-item **ratings**.
- A dataframe with **user characteristics**, such as gender, age, and occupation.
- A dataframe describing **movie features**, such as genre information.

Data wrangling was a crucial step, as the two datasets followed different formats and conventions. To build a unified and reusable recommendation pipeline, we standardized the structure of both datasets, allowing us to avoid adapting the code for each individual case.

Handling Format Differences

Several differences between the datasets required careful preprocessing. For instance:

- In the **1M dataset**, user occupations were encoded numerically through a mapping (e.g., 0 → **other**, 1 → **academic/educator**, etc.), while in the smaller dataset,

occupations were provided directly as strings (e.g., `engineer`, `artist`).

- The **genre encoding** also differed. In the 100k dataset, genres were represented as a vector of boolean values, where each position corresponded to a specific genre (e.g., the first position represented `Action`, and a value of 1 indicated that the movie belonged to that genre). In contrast, the 1M dataset used a single string listing all applicable genres (e.g., `"Action|Adventure|Sci-Fi"`). To ensure consistency, we transformed the genre representation in the 1M dataset into the same boolean vector format as used in the 100k dataset.

Ratings Representation

To facilitate further modeling steps, we extracted the ratings into two formats:

- A **tabular format** (a pandas DataFrame), where each row contains the `user_id`, `movie_id`, and `rating`.
- A **sparse matrix representation**, where rows correspond to users, columns to movies, and entries to the respective ratings. This matrix format enables efficient computation during collaborative filtering and matrix factorization tasks.

Overall, this unification and preprocessing phase ensured that both datasets could be seamlessly used within a common recommendation framework, improving code reusability and maintainability.

Step 1 – Matrix Factorization (ALS)

Matrix factorization, particularly through **Alternating Least Squares (ALS)**, is especially effective when dealing with sparse datasets, as is common in recommendation systems. The main goal is to approximate the original user-item interaction matrix R by decomposing it into the product of two lower-dimensional matrices:

$$R \approx U \times I^T$$

Here, $U \in \mathbb{R}^{n \times f}$ represents the **user latent feature matrix**, and $I \in \mathbb{R}^{m \times f}$ represents the **item (movie) latent feature matrix**, where n is the number of users, m the number of items, and f the number of latent factors.

Latent Feature Representation

The rows of the matrix U capture the hidden preferences of each user across a set of abstract features (e.g., preference for action films, romantic subplots, or indie productions), while the rows of matrix I encapsulate the corresponding hidden characteristics of each movie. These learned features are not explicitly labeled but are inferred from the interaction patterns present in the data.

These latent feature matrices play a dual role in our pipeline:

- They are used to reconstruct or predict the user-item rating matrix via:

$$\hat{R}_{ui} = U_u \cdot I_i^T$$

- They form the foundation for **computing similarities** between users and between items, which we will exploit in later stages to enhance recommendations and address the cold-start problem.

Regularization and Hyperparameter Tuning

To prevent overfitting—especially in the presence of sparse data—we introduced **L2 regularization** into the loss function. The optimization objective becomes:

$$\min_{U, I} \sum_{(u, i) \in \mathcal{K}} (R_{ui} - U_u \cdot I_i^T)^2 + L(\|U_u\|^2 + \|I_i\|^2)$$

where $\mathcal{K}$ is the set of observed (user, item) pairs and L is the regularization parameter.

The choice of L was critical. A small value may lead to overfitting, while a large value could cause underfitting. We employed a **grid search optimization** over a range of plausible values to select the best-performing regularization strength, based on a validation set.

K-Fold Cross-Validation in ALS

In order to ensure the robustness and generalization capability of our ALS (Alternating Least Squares) model, we employed **K-Fold Cross-Validation** as a validation technique.

K-Fold Cross-Validation is a resampling procedure used to evaluate machine learning models on a limited data sample. The procedure involves partitioning the original dataset into k equally (or nearly equally) sized folds. For each of the k iterations, one fold is retained as the validation set, and the remaining $k - 1$ folds are used as the training set. This process is repeated k times, with each fold used exactly once as the validation data. The performance metrics (RMSE) are then averaged over the k trials to provide a more reliable estimate of model performance.

This approach allows us to mitigate the risk of overfitting and to better estimate the model's performance on unseen data. Moreover, by validating on multiple subsets, we ensure that the ALS model is not overly sensitive to a particular train/test split, leading to more robust recommendations.

Limitations and Role of the ALS Output

The output of ALS—i.e., the predicted rating matrix $R_1 = \hat{R}$ —provided a valuable estimation of how each user might rate unseen movies. However, this was not the final step in our pipeline. ALS alone does not address the cold-start problem, where new users or new items lack interaction data. Therefore, although R_1 was not sufficient on its own, it served as a **core component** of our final recommendation strategy.

Step 2 – User Similarity-Based Prediction Model

In order to address the limitations of pure collaborative filtering—especially under cold-start conditions—we designed a hybrid approach based on user similarity. The intuition is that users with similar demographic profiles or preferences tend to rate items in comparable ways. To this end, we constructed multiple similarity matrices and combined them into a unified user similarity score.

User Similarity Matrices

We computed different matrices that quantify similarity between users from different perspectives:

- **Age Similarity Matrix** (S_{age}):

Each user's age was first mapped into predefined categorical bins as provided in the dataset (e.g., under 18, 18-24, 25-34, etc.). The similarity between two users a and b was then defined as:

$$S_{\text{age}}(a, b) = \frac{1}{|A(a) - A(b)| + 1}$$

where $A(a)$ and $A(b)$ are the categorical age levels of users a and b respectively. The $+1$ in the denominator avoids division by zero and normalizes the metric.

- **Gender Similarity Matrix** (S_{gender}):

A binary similarity matrix where:

$$S_{\text{gender}}(a, b) = \begin{cases} 1 & \text{if } G(a) = G(b) \\ 0 & \text{otherwise} \end{cases}$$

where $G(a)$ denotes the gender of user a .

- **Genre Preference Similarity Matrix** (S_{genre}):

For each user, we extracted the set of genres associated with movies they rated 4 or 5 stars—indicating strong preference. Using these sets, we computed the **Jaccard similarity** between users:

$$S_{\text{genre}}(a, b) = \frac{|F(a) \cap F(b)|}{|F(a) \cup F(b)|}$$

where $F(a)$ and $F(b)$ are the sets of preferred genres for users a and b .

Weighted User Similarity

We computed an aggregated user similarity matrix S_{dem} as a weighted sum of the three previously defined similarity matrices:

$$S_{\text{dem}} = L_1 S_{\text{age}} + L_2 S_{\text{gender}} + L_3 S_{\text{genre}}$$

Here, L_1, L_2, L_3 are tunable weights that determine the contribution of each similarity component.

Final Similarity Score

To further enhance our model, we incorporated the similarity derived from the **user latent feature matrix** U obtained from ALS. The final similarity matrix S_{final} was computed as:

$$S_{\text{final}} = \alpha \cdot S_{\text{dem}} + \beta \cdot S_{\text{ALS}}$$

where S_{ALS} is the cosine similarity matrix derived from the rows of U , and α, β are weighting parameters that control the balance between demographic similarity and latent preference similarity.

Prediction Formula

Given a user a and a movie i , the predicted rating $\hat{R}_{ai}$ was computed using a neighborhood-based approach:

- First, we identified the top 50 users b most similar to a according to S_{final} , and who have rated movie i .
- The predicted rating was then given by:

$$\hat{R}_{ai} = \bar{r}_a + \frac{\sum_{b \in N(a,i)} S_{\text{final}}(a, b) \cdot (r_{bi} - \bar{r}_b)}{\sum_{b \in N(a,i)} S_{\text{final}}(a, b)}$$

where:

- $\bar{r}_a$ is the average rating of user a ,
- r_{bi} is the rating of user b for movie i ,
- $\bar{r}_b$ is the average rating of user b ,
- $N(a, i)$ is the set of top 50 similar users to a who rated item i .

Hyperparameter Optimization with PSO

To determine the optimal values for the weights $L_1, L_2, L_3, \alpha, \beta$, we employed **Particle Swarm Optimization (PSO)**.

A Simple Explanation of PSO

PSO is a nature-inspired optimization algorithm that mimics the social behavior of birds flocking or fish schooling. Each possible solution (called a *particle*) "flies" through the solution space, adjusting its position based on:

- Its own experience (best value it has found so far),
- The experience of neighboring particles (global or local best).

Over time, the particles tend to converge toward the best-found region in the search space. PSO is especially effective for continuous optimization problems with multiple interacting parameters—like the one we are solving.

In our case, each particle represented a candidate set of weights $(L_1, L_2, L_3, \alpha, \beta)$, and the objective was to minimize the RMSE between the predicted and true ratings on a validation set.

Step 3 – Movie Similarity-Based Prediction Model

In order to address the item cold start problem, we designed a hybrid approach based on movie similarity. The intuition lies in the fact that if two movies are similar to each other, then if a user likes one of them, probably he also likes the other one. To achieve this, we constructed a single similarity movie matrix based on their descriptions.

Extracting the movies' descriptions

The items' descriptions are the baseline for all the predictions in this step. In order to obtain quality text, we used the Wikipedia API.

This library helped us in extracting only the useful information based on the Wikipedia page of the movie. We extracted the text from each section, except those in the following parts of the web page: "*See also*", "*References*", "*External links*", "*Notes*", "*Bibliography*". These sections mainly contain hyperlinks to other web pages and are not useful for our purposes.

Unfortunately, we did not manage to retain the descriptions for all movies. Some of them had the title in the dataset which was either wrong or had more or less uppercase letters than the one on Wikipedia. However, this number is a negligible percentage of the number of movies.

Log-Likelihood Weighting

After obtaining the movies' descriptions we can start working on the text. The first step in order to obtain a similarity matrix is the *Log-Likelihood (LL)*.

Unlike conventional techniques such as TF-IDF, LL uses a statistical approach to analyze each term's weight. Terms that appear too frequently in the corpus, will have lower LL values, on the contrary, characterizing words tend to have higher values.

The word frequency distribution is captured in the contingency table below:

	Corpus 1	Corpus 2	Total
Frequency of word	a	b	$a + b$
Frequency of other words	$c - a$	$d - b$	$c + d - a - b$
Total	c	d	$c + d$

To calculate the log-likelihood score, we first compute the expected frequencies under the

assumption of independence:

$$E_1 = c \cdot \frac{a+b}{c+d}, \quad E_2 = d \cdot \frac{a+b}{c+d}$$

where:

- a and b are the observed frequencies of the word in Corpus 1 and Corpus 2, respectively,
- c and d are the total word counts in Corpus 1 and Corpus 2, respectively,
- E_1 and E_2 are the expected frequencies of the word in Corpus 1 and Corpus 2 under the null hypothesis.

Then, the log-likelihood (LL) value is calculated as:

$$LL = 2 \cdot \left(a \cdot \ln \left(\frac{a}{E_1} \right) + b \cdot \ln \left(\frac{b}{E_2} \right) \right)$$

This statistic quantifies how much more likely the observed word distribution is compared to what would be expected by chance. A higher LL score indicates that the word is more discriminative between the two corpora.

Continuous Bag Of Words (CBOW)

Another step towards computing the item similarity matrix is to compute the CBOW representation of the words in our corpus.

This representation is used to address the problem that by using only LL to compute movie similarity, it could be affected by the sparsity of the movie vectors, degrading prediction performance.

CBOW is a Word2Vec model that predicts target words based on context words. For our implementation, we opted for an embedding of size 300 and a window size of 10. These parameters are very commonly used in Word2Vec to balance computing time and precision.

The Movie Vectors

Now that we have computed the log-likelihood and the CBOW representation, we can build the vectors that will represent our movies. To do so, we will use the following formula:

$$V_{movie} = \frac{\sum_{g=1}^n LL_{(a_g)} V_{Word2Vec(a_g)}}{\sum_{g=1}^n LL_{(a_g)}}$$

Item Similarity Matrix

We are now able to compute the similarity matrix. We will use two different metrics:

- **RV Coefficient:**

$$RV_{a,b} = \frac{\sum_{g=1}^s (a_g \cdot b_g)}{\sqrt{\sum_{g=1}^n (a_g)^2} * \sqrt{\sum_{g=1}^m (b_g)^2}}$$

where a_g is the g -th element of vector a and b_g is the g -th element of vector b

- **Cosine similarity:**

$$Sim_{(a,b)}HFM = \frac{\vec{a} \cdot \vec{b}}{||\vec{a}|| \cdot ||\vec{b}||}$$

In order to compute the overall similarity of two movies, we will need the movie vectors just found and the implicit movie feature vector matrix found in step 1:

$$Sim_{(i,j)} = \alpha_m Sim_{(i,j)}HFM_m + \beta_m RV_{(i,j)}$$

Where $Sim_{(i,j)}HFM_m$ denotes the cosine similarity between movies i and j in the hidden movie feature vector matrix; while $RV_{(i,j)}$ denotes the RV coefficient of the two movie vectors.

In order to find the best α and β we run evaluated various models using a validation set. The best values for the parameters are:

- $\alpha = 0.3$
- $\beta = 0.7$

Predicted Rating Matrix

The predicted rating matrix for Step 3 should be calculated following this formula:

$$\hat{R}_{a,i} = \bar{R}_i + \frac{\sum_{j \in Top-n} Sim_{i,j} (R_j - \bar{R}_j)}{\sum_{j \in Top-n} Sim_{i,j}}$$

Where:

- $\hat{R}_{a,i}$ is the predicted rating for user a on movie i .
- $\bar{R}_i$ is the average rating of movie i
- The sum in the numerator represents a weighted aggregation of rating deviations for the Top-N most similar movies.
- The denominator ensures normalization by dividing by the total similarity score.

Evaluation of the model

When testing the model on this step, we obtained a great RMSE, so we tried to find the cause of this problem. For every item-user couple to predict the rating of, we heavily rely on finding movies that are both similar to the one considered and that have been rated by the user.

Unfortunately, using these datasets and by dividing them in a train and test set, we do not always manage to find a sufficient number of data in order to make a reliable prediction. The TOP-N of this part is so computed:

```
for each movie do
  Find the 20 movies most similar to it
  # only 20 because then, we would consider also movies not similar to the said movie

  for each user do
    Find all the movies the user has rated
    Intersect the most similar movies with the items the user has rated and consider
    only 10 of them
    # this is the TOP-N, which maintains the ordering from most to least similar
  end for
end for
```

This means that for many movie-user couples we do not manage to find enough evidence to confidently predict all the rating matrix. Here are shown the results we obtain considering only those couples that had at least a fixed number of elements in their final TOP-N:

Minimum Elements in TOP-N	Percentage of Such Couples in MovieLens 100K Dataset	RMSE on Test Set
0	100%	2.51
1	88%	2.47
3	62%	2.41
5	56%	2.55
10	1%	0.94

As we can see, when we have a considerable number of elements used for making the prediction, the RMSE drops significantly.

It was also tried to broaden the considered similar movies when computing the TOP-N, but by doing so the algorithm would start considering non similar movies, worsening the predictions.

This model could be improved by using other features to compute the movie similarities, as genres, year of release, target audience...

In order to evaluate the final model, we will consider only those item-user pairs that provide sufficient data to make a good prediction.

Conclusion

This system integrates:

- **ALS** for latent preference extraction.
- **Demographic partitioning** for cold-start users.
- **Text + embedding-based item similarity** for new items.

To build the final prediction matrix, we exploit the formula:

$$R(a, i) = \alpha \hat{R}_{(a,i)}^{s1} + \beta \frac{\hat{R}_{(a,i)}^{s2} + \hat{R}_{(a,i)}^{s3}}{2}$$

In order to find the best parameter for α and β we employed grid-search optimization, ending up with:

Dataset	α	β	RMSE
100K	0.5	0.5	1.0158
1m	0.6	0.4	0.9189

Unfortunately, this result differs from the one found in the reference paper where: One

Dataset	α	β	RMSE
100K	0.3	0.7	0.881
1m	0.3	0.7	0.824

way to get closer to the paper's result would have been to find a way to enhance the performance of step-2 and step-3. Future enhancements may include advanced machine learning methods and refined similarity measures for better accuracy and engagement.